tty / console / uart 是怎么绑定起来的（启动注册 → devtmpfs 节点 → open 分派 → 会话绑定）

① 启动注册（kernel/init/main.c，单核早期、尚无进程）

main()
device_init(); ... consoleinit(); ttyinit(); printfinit();
init/main.c:64-69

↓

consoleinit()
initlock(&cons.lock, "cons")
platform_device_register(&uartdev)
platform_driver_register(&uartdrv)
tty/console.c:264

platform_device_register
dev->bus = &platform_bus
→ device_register(dev)
加入 devices[]，建 sysfs
遍历 drivers[] 试 bind：
此时还没有驱动 → 不匹配
devtmpfs_device_add(dev)
base/device.c:620,457

platform_driver_register
drv->bus = &platform_bus
→ driver_register(drv)
加入 drivers[]
遍历 devices[] 调 bind()
找到 uartdev ↓
base/device.c:627,558

↓

bind(dev, drv)
bus 相同 &&
platform_match():
strcmp(dev->name, drv->name)
"bcm2837-mini-uart" 两边相同
dev->driver = drv
base/device.c:58,30

↓

bus_probe → uart_probe
uartinit():
GPIO14/15 ALT5，8N1
baud=270，IER=RX 中断
initlock uart_tx / uart_hw
register_chrdev(CONSOLE,
{consoleread, consolewrite})
tty/console.c:271

ttyinit()
initlock(&ttyfg_lock)
register_chrdev(TTYS0,
{consoleread, consolewrite})
register_chrdev(TTY,
{ttycurrentread,
ttycurrentwrite})
device_register(&ttyS0_device)
device_register(&tty_device)
无 bus → 不 bind，只建
sysfs + devtmpfs 节点
tty/tty.c:96

② 注册完成后的内核对象

device_core.chrdevs[NDEV]（按 major 索引的 fops 表）

[1] CONSOLE	"console"	consoleread / consolewrite	← uart_probe
[2] TTY	"tty"	ttycurrentread / ttycurrentwrite	← ttyinit
[3] TTYS0	"ttyS0"	consoleread / consolewrite	← ttyinit

注：三个 major 最终都落到同一个 cons + uart，tty 只多一层 p->ctty 检查

struct device uartdev
name "bcm2837-mini-uart"
bus &platform_bus
driver &uartdrv ← bind 后
devnode "console" (1,0)
res MEM 0x3F215000 AUX
res MEM 0x3F200000 GPIO
res IRQ 29 (AUX)
class "tty"
console.c:102

struct device_driver uartdrv
name "bcm2837-mini-uart"
bus &platform_bus
probe uart_probe
console.c:120

platform_match：按名字配对

ttyS0_device / tty_device
devnode "ttyS0" (3,0)
devnode "tty" (2,0)
class "tty"，bus = 0
tty.c:98

devtmpfs（do_mounts.c）→ /dev
devtmpfs_init → devtmpfs_populate()：遍历 device_at(i)，
有 devnode 就 kmknod(T_DEVICE, dev_major, dev_minor)
/dev/console T_DEVICE major=1
/dev/tty T_DEVICE major=2
/dev/ttyS0 T_DEVICE major=3
之后的热插拔：devtmpfs_device_add → 事件队列 → devtmpfs_worker
sysfs：/sys/devices/bcm2837-mini-uart/ioaddr，/sys/class/tty/{...}
init/do_mounts.c:250,342

③ 硬件与中断绑定

BCM2837 AUX Mini UART
REGS_AUX @ PERIPHERAL_BASE+0x215000
mu_io / mu_lsr / mu_ier / mu_baud
GPIO14 TXD1, GPIO15 RXD1 (ALT5)
aux.h

↓

中断控制器（bcm2837.c）
gicv3init: ENABLE_IRQS_1 |= 1<<29
gic_iar(): IRQ_PENDING_1 & (1<<29)
→ 返回 UART0_IRQ
名字叫 gic_*，在 rpi3 上是 BCM 传统控制器
arch/arm64/bcm2837.c:57,97

↓

异常向量 → devintr()
trapasm.S: bl kernelirq / bl userirq
devintr(): irq == UART0_IRQ
→ uartintr()
这是"硬编码绑定"：没走 request_irq，
uartdev 的 IRQ resource 只做展示
core/trap.c:163

printf 绑定
printfinit(): pr.locking = 1
printf → consputc → uartputc_sync
不经过 chrdev 表，直接调 uart.c
lib/printf.c:133

④ 用户态把进程绑到终端：init → login → sh

init (pid 1)
open("/dev/ttyS0") → fd 0
dup(0) dup(0) → fd 1,2
ttyS0 打不开才退回 /dev/console
user/init.c:91

→

sys_open
namei → inode T_DEVICE
f->type = FD_DEVICE
f->major = ip->major (3)
chrdev_open(f):
f->f_op = chrdevs[3].fops
fs/sysfile.c:999

→

login
tty_attach(0) → ttyattach(0)
p->sid = p->pgid = pid
p->ctty = TTYS0
ttyfg_pgid = pid 🔒ttyfg_lock
user/login.c:69 tty.c:36

→

sh (login exec)
open("/dev/tty") 补齐 fd
→ chrdevs[2] = tty_fops
ttycurrentread: ctty==TTYS0?
ctty 经 fork 继承（proc.c:700）
user/sh.c:274

→

sh 执行命令
pid = fork1()
tty_set_foreground(pid)
→ ttyfg_pgid = pid
wait(0); 再设回自己
^C 只打到 ttyfg_pgid 这一组
user/sh.c:325-334

一句话总结三者关系

uart.c 是硬件驱动：只管 Mini UART 寄存器、32 B 发送环和中断，自身不暴露任何设备文件。

console.c 是终端本体：持有唯一的输入环 cons 和行规程，并以 platform driver 的身份 probe uartdev，在 probe 里初始化 uart 并注册 major 1（/dev/console）。

tty.c 是会话层：再注册 major 3（/dev/ttyS0，和 console 用同一组函数）和 major 2（/dev/tty，按 p->ctty 转发），并维护前台进程组 ttyfg_pgid。

绑定分三处：设备↔驱动靠 platform_match 按名字配对；设备文件↔函数靠 inode.major 查 chrdevs[] 表（chrdev_open 时拷进 f->f_op）；

进程↔终端靠 p->ctty / p->sid / p->pgid（login 的 tty_attach 设置，fork 继承），中断↔驱动则是 devintr 里写死的 if(irq == UART0_IRQ)。

对比 Linux：Linux 的 uart_register_driver + uart_add_one_port 会生成 tty_driver/tty_port，/dev/ttyS0 由 tty_register_device 动态创建，

/dev/console 是 major 5 指向当前 console= 的那个 tty，/dev/tty 也是 major 5（minor 0）按 current->signal->tty 转发；中断由驱动 request_irq 绑定。